Analytical Problem Solving

Warehouse Delivery Robot – Intelligent Agent and Search Problem

Given:

This problem considers a delivery robot operating in a 5 × 5 warehouse grid. The robot starts at cell (1,1) and must deliver a package to the destination cell (5,5). Some cells are blocked because of shelving, and the robot cannot move through these cells.

The robot can move only in four directions: Up, Down, Left, and Right. Diagonal movement is not allowed. Each movement has a uniform cost of 1.

The blocked cells are:

- (2,2)
- (2,3)
- (3,3)
- (4,2)

The problem is solved using intelligent-agent modeling, Breadth-First Search (BFS), and Depth-First Search (DFS).

Problem Representation

The warehouse can be represented using the following 5 × 5 grid.

Row / Column	**1**	**2**	**3**	**4**	**5**
1	S	Free	Free	Free	Free
2	Free	X	X	Free	Free
3	Free	Free	X	Free	Free
4	Free	X	Free	Free	Free
5	Free	Free	Free	Free	G

S= Starting Position (1,1)

G= Goal Position (5,5)

X=Blocked cell

The robot has to find a valid path from S to G without entering any blocked cell.

A) PEAS Description of the Intelligent Agent

PEAS stands for:

- P – Performance Measure
- E – Environment
- A – Actuators
- S – Sensors

PEAS is used to describe the task environment in which an intelligent agent operates.

PEAS Description

PEAS Component	**Description**
Performance Measure	Successfully deliver the package to cell (5,5), avoid blocked cells, minimize the number of movements, and minimize total path cost.
Environment	A 5 × 5 warehouse grid containing free cells and blocked cells caused by shelving.
Actuators	The robot's movement mechanisms that allow it to move Up, Down, Left, or Right. The robot can also perform the delivery action at the destination.
Sensors	Sensors provide the robot with its current position and information about nearby cells, obstacles, and the destination.

Environment Characteristics

The warehouse environment can also be classified according to the following properties:

Property	**Classification**	**Explanation**
Number of agents	Single-agent	Only one robot is solving the problem.
Observability	Fully observable	The warehouse layout and blocked cells are known.
Determinism	Deterministic	A movement always produces the expected result if it is legal.
Environment	Static	The blocked cells and warehouse layout do not change during the search.
State space	Discrete	The robot occupies one discrete grid cell at a time.

Actions	Four actions	Up, Down, Left, and Right.
Step cost	Uniform	Every legal movement has cost 1.

The robot's main objective is to deliver the package to the destination. Therefore, the performance measure mainly depends on whether the robot reaches (5,5) successfully and how efficiently it gets there.

The environment is fully observable and deterministic because the robot knows the grid and the locations of the blocked cells. The robot's actuators are responsible for movement, while its sensors provide information about its current location and surroundings.

B) Classification of the Intelligent Agent

The most suitable type of intelligent agent for this problem is a:

Goal-Based Agent

A goal-based agent selects actions based on a desired goal state.

In this problem, the goal is:

Reach cell (5,5) and deliver the package.

The robot must plan a sequence of movements that takes it from (1,1) to (5,5) while avoiding the blocked cells.

Comparison of Agent Types

Agent Type	**Suitability**	**Reason**
Simple Reflex Agent	Not suitable	It mainly reacts to the current percept and does not plan a complete route toward the destination.
Model-Based Agent	Possible	It can maintain an internal model of the environment, but the problem mainly requires goal-directed path planning.
Goal-Based Agent	Best suited	It selects actions with the specific objective of reaching the destination.
Utility-Based Agent	Not necessary	Utility functions are useful when several objectives or preferences need to be compared. This problem mainly has one goal with uniform movement costs.

A goal-based agent is the best choice because the robot has a clearly defined goal state, which is (5,5). The robot needs to determine a sequence of actions that will reach this state while avoiding obstacles.

For example, the robot may choose:

Down → Down → Down → Down → Right → Right → Right → Right

This sequence reaches the goal while avoiding the blocked cells.

Therefore:

The warehouse delivery robot is best modeled as a goal-based intelligent agent.

C) Formal Search Problem

The warehouse problem can be formulated as a classical search problem using five components:

1. Initial State
2. Goal State
3. Actions
4. Transition Model
5. Path Cost

Search Problem Definition

Component	**Definition**
Initial State	(1,1)
Goal State	(5,5)
Actions	Up, Down, Left, Right
Transition Model	A legal movement changes the robot's current cell to the adjacent free cell. Movement outside the grid or into a blocked cell is not allowed.
Path Cost	Every movement costs 1. Therefore, total path cost equals the number of movements.

Initial State

The robot initially starts at = (1,1)

Goal State

The required destination is: (5,5)

Actions

The robot has four possible actions:

A=(Up,Down,Right,Left)

However, an action is allowed only if the resulting cell:

- is inside the 5 × 5 grid, and
- is not blocked.

Transition Model

If the robot is currently at: (r,c)

then the possible new positions are:

$$(r, -1c),\ (r, +1c),\ (r, c-1),\ (r, c+1)$$

provided that the resulting position is valid and not blocked.

Path Cost

Each movement has a cost of 1.

Therefore:

$$\text{Path Cost} = \text{Number of Moves}$$

For example, if the robot makes 8 movements:

$$\text{Cost} = 8 \times 1 = 8$$

D) Breadth-First Search (BFS)

BFS Concept

Breadth-First Search (BFS) explores the search space level by level. It first explores all states at depth 1, then all states at depth 2, then depth 3, and so on.

BFS uses a Queue, which follows the FIFO (First-In, First-Out) principle.

Since every movement in this problem has the same cost of 1, BFS is guaranteed to find the shortest path.

For the following expansion, the movement order is taken as:

Up → Down → Left → Right

Already visited cells are not inserted into the queue again.

BFS Step-by-Step Expansion

Step	Expanded Node	New Nodes Added	Queue / Frontier After Expansion
1	(1,1)	(2,1), (1,2)	(2,1), (1,2)
2	(2,1)	(3,1)	(1,2), (3,1)
3	(1,2)	(1,3)	(3,1), (1,3)
4	(3,1)	(4,1), (3,2)	(1,3), (4,1), (3,2)
5	(1,3)	(1,4)	(4,1), (3,2), (1,4)
6	(4,1)	(5,1)	(3,2), (1,4), (5,1)
7	(3,2)	—	(1,4), (5,1)
8	(1,4)	(2,4), (1,5)	(5,1), (2,4), (1,5)
9	(5,1)	(5,2)	(2,4), (1,5), (5,2)
10	(2,4)	(3,4), (2,5)	(1,5), (5,2), (3,4), (2,5)
11	(1,5)	—	(5,2), (3,4), (2,5)
12	(5,2)	(5,3)	(3,4), (2,5), (5,3)
13	(3,4)	(4,4), (3,5)	(2,5), (5,3), (4,4), (3,5)
14	(2,5)	—	(5,3), (4,4), (3,5)
15	(5,3)	(4,3), (5,4)	(4,4), (3,5), (4,3), (5,4)
16	(4,4)	(4,5)	(3,5), (4,3), (5,4), (4,5)
17	(3,5)	—	(4,3), (5,4), (4,5)
18	(4,3)	—	(5,4), (4,5)
19	(5,4)	(5,5)	(4,5), (5,5)
20	(4,5)	—	(5,5)
21	(5,5)	Goal reached	—

Path Representation

(1,1)

↓

(2,1)

↓

(3,1)

↓

(4,1)

↓

(5,1) → (5,2) → (5,3) → (5,4) → (5,5)

BFS Cost

There are 8 movements.

BFS Cost is 8

BFS Result

BFS successfully reaches the destination with a minimum cost of 8. Because all movements have equal cost, this is the optimal path.

E)Depth-First Search (DFS)

DFS Concept

Depth-First Search (DFS) explores one branch as deeply as possible before backtracking and exploring another branch.

DFS uses a Stack, which follows the LIFO (Last-In, First-Out) principle.

Unlike BFS, DFS does not guarantee the shortest path.

For this demonstration, the neighboring cells are considered in the order:

Up → Down → Right → Left

Since DFS uses a stack, the last inserted valid node is explored first.

DFS Step-by-Step Expansion

Step	Expanded Node	New Nodes Added	Stack After Expansion
1	(1,1)	(2,1), (1,2)	(1,2), (2,1)
2	(1,2)	(1,3)	(1,3), (2,1)
3	(1,3)	(1,4)	(1,4), (2,1)
4	(1,4)	(2,4), (1,5)	(1,5), (2,4), (2,1)
5	(1,5)	(2,5)	(2,5), (2,4), (2,1)
6	(2,5)	(3,5)	(3,5), (2,4), (2,1)

7	(3,5)	(3,4), (4,5)	(3,4), (4,5), (2,4), (2,1)
8	(3,4)	(4,4)	(4,4), (4,5), (2,4), (2,1)
9	(4,4)	(5,4), (4,3)	(4,3), (5,4), (4,5), (2,4), (2,1)
10	(4,3)	(5,3)	(5,3), (5,4), (4,5), (2,4), (2,1)
11	(5,3)	(5,2)	(5,2), (5,4), (4,5), (2,4), (2,1)
12	(5,2)	(5,1)	(5,1), (5,4), (4,5), (2,4), (2,1)
13	(5,1)	(4,1)	(4,1), (5,4), (4,5), (2,4), (2,1)
14	(4,1)	(3,1)	(3,1), (5,4), (4,5), (2,4), (2,1)
15	(3,1)	(3,2)	(3,2), (5,4), (4,5), (2,4), (2,1)
16	(3,2)	—	(5,4), (4,5), (2,4), (2,1)
17	(5,4)	(5,5)	(5,5), (4,5), (2,4), (2,1)
18	(5,5)	Goal reached	—

Path Representation

(1,1) → (1,2) → (1,3) → (1,4) → (1,5)

↓

(2,5)

↓

(3,5)

↓

(3,4)

↓

(4,4)

↓

(5,4) → (5,5)

DFS Cost

The path contains 10 movements.

$$\text{Cost} = 10 \times 1$$

DFS Cost= 10

DFS Result

DFS successfully reaches the goal, but the path found is longer than the path found by BFS.

Therefore, for this particular search order:

DFS is greater than BFS

DFS's exact path can change depending on the order in which neighboring cells are explored. However, DFS in general does not guarantee an optimal path.

F) Comparison of BFS and DFS

BFS and DFS are both uninformed search algorithms, but they behave differently.

Comparison Table

Feature	**BFS**	**DFS**
Data Structure	Queue	Stack
Principle	Explores level by level	Explores one branch deeply
Complete	Yes, for a finite branching problem	Yes for this finite grid when visited states are maintained
Optimal	Yes, when all step costs are equal	No
Time Complexity	$O(b^d)$	$O(b^m)$
Space Complexity	$O(b^d)$	$O(bm)$
Path obtained here	8 moves	10 moves
Path Cost	8	10
Memory Requirement	Higher	Lower
Main Advantage	Finds shortest path	Uses less memory
Main Disadvantage	Requires more memory	May find a longer path
Suitability here	Highly suitable	Less suitable

Where:

- b = branching factor
- d = depth of the shallowest solution

- m = maximum depth of the search tree

BFS is Better :

The warehouse robot has a uniform movement cost:

$$\text{Cost of every movement} = 1$$

Therefore, minimizing the total cost is equivalent to minimizing the number of movements.

BFS explores the search space level by level. It therefore reaches the goal at the shallowest possible depth.

In this problem:

$$\text{BFS Cost} = 8$$

while:

$$\text{DFS Cost} = 10$$

Thus, BFS produces a more efficient route.

DFS could potentially find a solution faster in some search spaces and uses less memory, but it may follow an unnecessarily long path before reaching the destination.

Final Conclusion

For the warehouse delivery robot, BFS is more suitable than DFS because:

1. The movement costs are uniform.
2. The objective is to find a minimum-cost path.
3. BFS is complete.
4. BFS guarantees the shortest path under uniform step costs.
5. BFS finds an 8-move path in this problem.
6. DFS finds a 10-move path with the chosen search order.

Therefore: BFS is the preferred search algorithm for this warehouse problem

Program:

q.py - C:/Users/bhuvi/Downloads/q.py (3.11.9)

File Edit Format Run Options Window Help

```
from collections import deque

start = (1, 1)
goal = (5, 5)
blocked = {
    (2, 2),
    (2, 3),
    (3, 3),
    (4, 2)
}

moves = [
    (-1, 0),
    (1, 0),
    (0, -1),
    (0, 1)
]

def neighbors(position):
    r, c = position
    for dr, dc in moves:
        nr = r + dr
        nc = c + dc
        if 1 <= nr <= 5 and 1 <= nc <= 5 and (nr, nc) not in blocked:
            yield (nr, nc)

def get_path(parent, goal):
    path = []
    node = goal
    while node is not None:
        path.append(node)
        node = parent[node]
    return path[::-1]

# ----------------- BFS -----------------
def bfs():
    queue = deque([start])
    visited = {start}
    parent = {start: None}
```

q.py - C:/Users/bhuvi/Downloads/q.py (3.11.9)

File Edit Format Run Options Window Help

```
    while queue:
        current = queue.popleft()
        if current == goal:
            return get_path(parent, goal)
        for nxt in neighbors(current):
            if nxt not in visited:
                visited.add(nxt)
                parent[nxt] = current
                queue.append(nxt)

# ----------------- DFS -----------------
def dfs():
    stack = [start]
    visited = {start}
    parent = {start: None}

    while stack:
        current = stack.pop()
        if current == goal:
            return get_path(parent, goal)
        for nxt in neighbors(current):
            if nxt not in visited:
                visited.add(nxt)
                parent[nxt] = current
                stack.append(nxt)
```

```
*q.py - C:/Users/bhuvi/Downloads/q.py (3.11.9)*
File  Edit  Format  Run  Options  Window  Help
# Execute BFS
bfs_path = bfs()
print("BFS Path:")
print(bfs_path)
print("BFS Cost:", len(bfs_path) - 1)

# Execute DFS
dfs_path = dfs()
print("\nDFS Path:")
print(dfs_path)
print("DFS Cost:", len(dfs_path) - 1)
```

Output:

```
IDLE Shell 3.11.9
File  Edit  Shell  Debug  Options  Window  Help
Python 3.11.9 (tags/v3.11.9:de54cf5, Apr  2 2024, 10:12:12) [MSC v.1938 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
================ RESTART: C:/Users/bhuvi/Downloads/q.py ================
BFS Path:
[(1, 1), (2, 1), (3, 1), (4, 1), (5, 1), (5, 2), (5, 3), (5, 4), (5, 5)]
BFS Cost: 8

DFS Path:
[(1, 1), (1, 2), (1, 3), (1, 4), (1, 5), (2, 5), (3, 5), (3, 4), (4, 4), (5, 4), (5, 5)]
DFS Cost: 10
>>>
```

Final Results Table

Algorithm	Path Length	Total Cost	Optimal?
BFS	8 moves	8	Yes
DFS	10 moves	10	No

Hence, BFS is the most suitable algorithm for this warehouse delivery problem because it guarantees the shortest path when all movements have equal cost.

Conclusion

The warehouse delivery problem was successfully modeled as an intelligent-agent and search problem. The robot was represented using the PEAS framework, and a goal-based agent was identified as the most suitable agent type.

The search problem was formally defined using the initial state, goal state, actions, transition model, and path cost. BFS and DFS were then applied to the warehouse grid.